

# RealtimeKit Swift Package 开发提示词

## 项目概述

开发一个名为 **RealtimeKit** 的 Swift Package，用于集成多家第三方 RTM (Real-Time Messaging) 和 RTC (Real-Time Communication) 服务提供商，为 iOS/macOS 应用提供统一的实时通信解决方案。

## 核心目标

- 提供统一的 API 接口，屏蔽不同服务商的差异
- 支持主流 RTC/RTM 提供商（声网Agora、腾讯云、即构ZEGO等）
- 同时兼容 **UIKit** 和 **SwiftUI** 调用方式
- 插件化架构，便于扩展新的服务商
- 高性能、低延迟的实时通信体验
- 完善的错误处理和状态管理

## 技术要求

### Swift 版本支持

- Swift 6.0+
- iOS 13.0+
- macOS 10.15+

## 架构设计原则

1. **Protocol-Oriented Programming** - 使用协议定义统一接口
2. **Dependency Injection** - 支持依赖注入，便于测试
3. **Plugin Architecture** - 插件化设计，支持动态加载
4. **Async/Await** - 全面支持现代 Swift 异步编程
5. **Memory Safety** - 避免内存泄漏，合理管理资源
6. **Modular Design** - 模块化架构，可直接封装为 Swift Package

## 核心功能模块

### 1. 核心抽象层 (Core)

```
swift
```

// 主要协议定义

```
public protocol RTCTProvider {  
    func initialize(config: RTCTConfig) async throws  
    func createRoom(roomId: String) async throws -> RTCRoom  
    func joinRoom(roomId: String, userId: String) async throws  
    func leaveRoom() async throws  
  
    // 转推流功能  
    func startStreamPush(config: StreamPushConfig) async throws  
    func stopStreamPush() async throws  
    func updateStreamPushLayout(layout: StreamLayout) async throws
```

// 继媒体流功能

```
    func startMediaRelay(config: MediaRelayConfig) async throws  
    func stopMediaRelay() async throws  
    func updateMediaRelayChannels(config: MediaRelayConfig) async throws  
    func pauseMediaRelay(toChannel: String) async throws  
    func resumeMediaRelay(toChannel: String) async throws
```

// Token 管理

```
    func renewToken(_ newToken: String) async throws  
    func onTokenWillExpire(_ handler: @escaping (Int) -> Void) // 秒数倒计时
```

}

```
public protocol RTMProvider {  
    func initialize(config: RTMConfig) async throws  
    func sendMessage(_ message: RealtimeMessage) async throws  
    func subscribe(to channel: String) async throws
```

// 消息处理功能

```
    func setMessageHandler(_ handler: @escaping (RealtimeMessage) -> Void)  
    func setConnectionStateHandler(_ handler: @escaping (ConnectionState) -> Void)  
    func processIncomingMessage(_ rawMessage: Any) async throws -> RealtimeMessage
```

// Token 管理

```
    func renewToken(_ newToken: String) async throws  
    func onTokenWillExpire(_ handler: @escaping (Int) -> Void)
```

}

// 新增转推流配置

```
public struct StreamPushConfig {  
    let pushUrl: String  
    let width: Int
```

```
    let height: Int
    let bitrate: Int
    let frameRate: Int
    let layout: StreamLayout
}

public struct StreamLayout {
    let backgroundColor: UInt32
    let regions: [StreamRegion]
}

public struct StreamRegion {
    let uid: String
    let x: Double
    let y: Double
    let width: Double
    let height: Double
    let zOrder: Int
    let alpha: Double
}

// 新增继媒体流配置
public struct MediaRelayConfig {
    let sourceChannel: MediaRelayChannelInfo
    let destinationChannels: [MediaRelayChannelInfo]
    let relayMode: MediaRelayMode
}

public struct MediaRelayChannelInfo {
    let channelName: String
    let token: String?
    let uid: UInt32
    let serverUrl: String?
}

public enum MediaRelayMode {
    case oneToOne    // 1对1中继
    case oneToMany    // 1对多中继
    case manyToMany    // 多对多中继
    case crossChannel // 跨频道中继
}

public struct MediaRelayState {
    let state: RelayState
}
```

```
let sourceChannel: String
let destinationChannels: [String: ChannelRelayState]
}

public enum RelayState {
    case idle
    case connecting
    case running
    case failure(MediaRelayError)
}

public enum ChannelRelayState {
    case connected
    case disconnected
    case failure(MediaRelayError)
}

public enum MediaRelayError: Error {
    case serverConnectionLost
    case serverNoResponse
    case invalidToken
    case userNotInChannel
    case destinationChannelFull
    case networkError(String)
}
```

## 2. 统一管理器 (Manager)

swift

```

public class RealtimeManager {
    public static let shared = RealtimeManager()

    public func configure(provider: ProviderType, config: RealtimeConfig)
    public func switchProvider(to: ProviderType) async throws
    public func getCurrentProvider() -> ProviderType

    // Token 管理
    public func setupTokenRenewal(handler: @escaping (ProviderType) async -> String)
    public func renewAllTokens() async throws

    // 转推流管理
    public func startLiveStreaming(config: StreamPushConfig) async throws
    public func stopLiveStreaming() async throws
    public func updateStreamLayout(_ layout: StreamLayout) async throws

    // 继媒体流管理
    public func startMediaRelay(config: MediaRelayConfig) async throws
    public func stopMediaRelay() async throws
    public func updateMediaRelayChannels(config: MediaRelayConfig) async throws
    public func pauseMediaRelayChannel(_ channel: String) async throws
    public func resumeMediaRelayChannel(_ channel: String) async throws
    public func getMediaRelayState() -> MediaRelayState?

    // 消息处理中心
    public func setGlobalMessageHandler(_ handler: @escaping (RealtimeMessage) -> Void)
    public func registerMessageProcessor(_ processor: MessageProcessor)
}

```

// Token 管理器

```

public class TokenManager {
    public func scheduleTokenRenewal(provider: ProviderType, expiresIn: Int)
    public func handleTokenExpiration(provider: ProviderType) async throws
    public func isTokenExpiring(within seconds: Int) -> Bool
}

```

// 消息处理器协议

```

public protocol MessageProcessor {
    func canProcess(_ message: RealtimeMessage) -> Bool
    func process(_ message: RealtimeMessage) async throws -> ProcessedMessage?
}

```

// 继媒体流管理器

```

public class MediaRelayManager {
    public func startRelay(from source: MediaRelayChannelInfo, to destinations: [MediaRelayChannelInfo]) async throws
    public func addDestinationChannel(_ channel: MediaRelayChannelInfo) async throws
    public func removeDestinationChannel(_ channelName: String) async throws
    public func updateChannelToken(_ channelName: String, token: String) async throws
    public func getRelayStatistics() -> MediaRelayStatistics
}

// 继媒体流统计信息
public struct MediaRelayStatistics {
    let totalRelayTime: TimeInterval
    let bytesSent: UInt64
    let bytesReceived: UInt64
    let packetsLost: UInt32
    let averageDelay: TimeInterval
    let channelStatistics: [String: ChannelStatistics]
}

public struct ChannelStatistics {
    let channelName: String
    let connectionState: ChannelRelayState
    let bytesSent: UInt64
    let bytesReceived: UInt64
    let lastUpdateTime: Date
}

```

### 3. 消息系统 (Messaging)

- 文本消息
- 富媒体消息（图片、音频、视频）
- 自定义消息格式
- 消息历史管理
- 离线消息处理

### 5. UI 集成层 (UI Integration)

swift

```

// UIKit 支持
public class RealtimeViewController: UIViewController {
    public func configureRealtime(provider: ProviderType)
    public func presentVideoCall(userId: String)
    public func showMessageInput() -> RealtimeMessageInputView
}

// SwiftUI 支持
@available(iOS 13.0, *)
public struct RealtimeView: View {
    @StateObject private var viewModel = RealtimeViewModel()

    var body: some View {
        // SwiftUI 组件实现
    }
}

public struct VideoCallView: View {
    // SwiftUI 视频通话组件
}

```

## 支持的服务商

### 第一批集成

1. 声网 **Agora**
  - Agora RTC SDK
  - Agora RTM SDK
2. 腾讯云 **TRTC**
  - TRTC SDK
  - TIM SDK
3. 即构 **ZEGO**
  - ZEGO Express SDK
  - ZEGO ZIM SDK

### 扩展支持

- 融云 RongCloud
- 网易云信 NeteaseIM

- 环信 Easemob
- 自定义服务商接入

## 模块化项目结构

## RealtimeKit/

- └── Sources/
  - └── RealtimeCore/ # 核心模块
    - └── Protocols/
      - └── RTCProvider.swift
      - └── RTMPProvider.swift
      - └── MessageProcessor.swift
    - └── Models/
      - └── StreamPushConfig.swift
      - └── StreamLayout.swift
      - └── MediaRelayConfig.swift
      - └── MediaRelayState.swift
      - └── TokenInfo.swift
    - └── Managers/
      - └── RealtimeManager.swift
      - └── TokenManager.swift
      - └── MessagePipeline.swift
      - └── MediaRelayManager.swift
    - └── Utils/
      - └── Extensions/
  - └── RealtimeUIKit/ # UIKit UI 模块
    - └── ViewControllers/
      - └── StreamingViewController.swift
      - └── MediaRelayViewController.swift
      - └── TokenSettingsViewController.swift
    - └── Views/
      - └── StreamLayoutView.swift
      - └── MediaRelayControlView.swift
      - └── MessageProcessorView.swift
    - └── Extensions/
    - └── Resources/
  - └── RealtimeSwiftUI/ # SwiftUI UI 模块
    - └── Views/
      - └── StreamingView.swift
      - └── MediaRelayView.swift
      - └── TokenManagementView.swift
      - └── MessageHandlerView.swift
    - └── ViewModels/
      - └── StreamingViewModel.swift
      - └── MediaRelayViewModel.swift
      - └── TokenViewModel.swift
    - └── Modifiers/
    - └── Extensions/

```
| |—— Providers/          # 服务商模块
| | |—— RealtimeAgora/
| | | |—— AgoraRTCPProvider.swift
| | | |—— AgoraRTMPProvider.swift
| | | |—— AgoraStreamPush.swift
| | | |—— AgoraMediaRelay.swift
| | | |—— AgoraTokenHandler.swift
| | | |—— AgoraMessageProcessor.swift
| | |—— RealtimeTencent/
| | | |—— TRTCTProvider.swift
| | | |—— TIMProvider.swift
| | | |—— TencentStreamPush.swift
| | | |—— TencentMediaRelay.swift
| | | |—— TencentTokenHandler.swift
| | |—— RealtimeZego/
| | | |—— ZegoExpressProvider.swift
| | | |—— ZegoZIMProvider.swift
| | | |—— ZegoStreamPush.swift
| | | |—— ZegoMediaRelay.swift
| | | |—— ZegoTokenHandler.swift
| |—— RealtimeMocking/      # 测试工具模块
| | |—— MockProviders/
| | |—— MockStreamPush.swift
| | |—— MockMediaRelay.swift
| | |—— MockTokenManager.swift
| | |—— TestHelpers/
| | |—— Fixtures/
| |—— RealtimeKit/          # 主聚合模块
| | |—— RealtimeKit.swift
| |—— UIKitDemo/           # UIKit Demo 应用
| | |—— App/
| | |—— ViewControllers/
| | | |—— LiveStreamingViewController.swift
| | | |—— MediaRelayViewController.swift
| | | |—— MessageCenterViewController.swift
| | |—— Views/
| | |—— Models/
| | |—— Resources/
| |—— SwiftUIDemo/         # SwiftUI Demo 应用
| | |—— App/
| | |—— Views/
| | |—— LiveStreamingView.swift
| | |—— MediaRelayView.swift
| | |—— MessageCenterView.swift
```

```
| |—— ViewModels/
| |—— Models/
| |—— Resources/
|—— Tests/
| |—— RealtimeKitTests/
| |—— TokenTests/
| |—— StreamPushTests/
| |—— MediaRelayTests/
| |—— MessageProcessingTests/
|—— Package.swift
|—— README.md
|—— Documentation/
```

## Demo 应用功能要求

### UIKit Demo 应用特性

swift

```
// UIKitDemo/App/AppDelegate.swift
import UIKit
import RealtimeKit
import RealtimeUIKit
import RealtimeAgora

@main
class AppDelegate: UIResponder, UIApplicationDelegate {
    func application(_ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        // 配置 RealtimeKit
        RealtimeManager.shared.configure(
            provider: .agora,
            config: RealtimeConfig(
                appId: "demo_app_id",
                appSecret: "demo_secret"
            )
        )
        return true
    }
}
```

### 核心功能演示:

- 登录/注册界面

- 房间列表和创建
- 文本消息聊天
- 1v1 视频通话
- 多人会议室
- 直播转推流设置
- 媒体流跨频道中继
- Token 自动续期演示
- 消息处理中心
- 设置和配置界面

## SwiftUI Demo 应用特性

swift

```
// SwiftUIDemo/App/App.swift
import SwiftUI
import RealtimeKit
import RealtimeSwiftUI
import RealtimeAgora

@main
struct RealtimeSwiftUIDemoApp: App {
    init() {
        // 配置 RealtimeKit
        RealtimeManager.shared.configure(
            provider: .agora,
            config: RealtimeConfig(
                appId: "demo_app_id",
                appSecret: "demo_secret"
            )
        )
    }

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

## 核心功能演示:

- 现代 SwiftUI 界面设计
- 声明式 UI 状态管理
- Combine 数据流处理
- 原生 SwiftUI 动画效果
- iPad 自适应布局
- 实时转推流控制面板
- 跨频道媒体流中继界面
- **Token** 状态实时监控
- 消息处理可视化界面

## Demo 应用权限配置

相机权限: "This demo app uses the camera for video calls." 麦克风权限: "This demo app uses the microphone for audio calls."

相册权限: "This demo app allows selecting images to share." 后台音频: "This demo app supports background audio for calls."

## 运行和测试方式

```
bash

# 运行 UIKit Demo
swift run UIKitDemoApp

# 运行 SwiftUI Demo
swift run SwiftUIDemoApp

# 在 Xcode 中打开
open Package.swift
# 然后选择对应的 Scheme 运行
```

## 模块化使用方式

### 1. 完整导入 (推荐)

```
swift
```

```
// 导入完整功能
import RealtimeKit

// 自动包含所有模块: Core + UIKit + SwiftUI
let manager = RealtimeManager.shared
```

## 2. 按需导入 (精简)

```
swift

// 仅导入核心功能
import RealtimeCore

// 仅导入特定 UI 框架
import RealtimeUIKit // 或
import RealtimeSwiftUI

// 仅导入特定服务商
import RealtimeAgora
import RealtimeTencent
```

## 3. 测试环境导入

```
swift

// 导入测试工具
import RealtimeMocking

let mockProvider = MockRTCProvider()
```

## 4. Package.swift 依赖配置

```
swift
```

```
// 在其他项目中使用
dependencies: [
    // 完整功能
    .package(url: "https://github.com/yourorg/RealtimeKit", from: "1.0.0")
],
targets: [
    .target(
        name: "YourApp",
        dependencies: [
            // 可选择性导入
            .product(name: "RealtimeCore", package: "RealtimeKit"),
            .product(name: "RealtimeSwiftUI", package: "RealtimeKit"),
            .product(name: "RealtimeAgora", package: "RealtimeKit")
        ]
    )
]
```

## 配置系统设计

### 配置文件支持

```
swift

public struct RealtimeConfig {
    let appId: String
    let appSecret: String
    let serverUrl: String?
    let region: Region
    let logLevel: LogLevel
    let features: [Feature]
}

// 支持多种配置方式
// 1. 代码配置
// 2. plist 文件配置
// 3. JSON 配置文件
// 4. 环境变量配置
```

## API 设计原则

### 1. 链式调用支持

```
swift
```

### RealtimeKit

```
.configure(provider: .agora, config: config)
.enableLogging(.debug)
.setDelegate(self)
.enableAutoTokenRenewal(true)
.setTokenRenewalHandler { provider in
    // 获取新 Token 的逻辑
    return await TokenService.renewToken(for: provider)
}
.enableStreamPush()
.enableMediaRelay()
.setMessageProcessor(CustomMessageProcessor())
```

## 2. Result 类型错误处理

swift

```
public enum RealtimeError: Error {
    case configurationError(String)
    case connectionFailed(String)
    case authenticationFailed
    case networkError(Error)

    // Token 相关错误
    case tokenExpired(ProviderType)
    case tokenRenewalFailed(ProviderType, Error)
    case invalidToken(ProviderType)

    // 转推流相关错误
    case streamPushStartFailed(String)
    case streamPushStopFailed(String)
    case invalidStreamConfig(String)
    case streamLayoutUpdateFailed(String)

    // 继媒体流相关错误
    case mediaRelayStartFailed(String)
    case mediaRelayStopFailed(String)
    case mediaRelayUpdateFailed(String)
    case invalidRelayConfig(String)
    case relayChannelConnectionFailed(String)

    // 消息处理相关错误
    case messageProcessingFailed(String)
    case unsupportedMessageType(String)
    case messageHandlerNotFound
}
```

```
// 新增状态枚举
public enum StreamPushState {
    case stopped
    case starting
    case running
    case stopping
    case failed(Error)
}
```

```
public enum ProcessingState {
    case idle
    case processing
    case completed
}
```

```
case failed(Error)
}

public struct TokenWarning {
    let provider: ProviderType
    let expiresIn: Int // 秒数
    let renewalAvailable: Bool
}
```

## 4. 多种回调方式支持

swift

// UIKit - Delegate 模式

```
public protocol RealtimeDelegate: AnyObject {  
    func didReceiveMessage(_ message: RealtimeMessage)  
    func didJoinRoom(_ roomId: String)  
  
    // 新增 Token 处理  
    func tokenWillExpire(provider: ProviderType, in seconds: Int)  
    func tokenDidRenew(provider: ProviderType)  
  
    // 新增转推流状态  
    func streamPushDidStart()  
    func streamPushDidStop()  
    func streamPushDidFail(error: RealtimeError)  
  
    // 新增继媒体流状态  
    func mediaRelayDidStart()  
    func mediaRelayDidStop()  
    func mediaRelayDidFail(error: MediaRelayError)  
    func mediaRelayChannelDidConnect(_ channel: String)  
    func mediaRelayChannelDidDisconnect(_ channel: String, error: MediaRelayError?)  
  
    // 新增消息处理状态  
    func messageProcessingDidStart()  
    func messageDidProcess(_ message: ProcessedMessage)  
    func messageProcessingDidFail(_ error: Error)  
}
```

// UIKit - Closure 模式

```
public func onMessageReceived(_ handler: @escaping (RealtimeMessage) -> Void)  
public func onTokenWillExpire(_ handler: @escaping (ProviderType, Int) -> Void)  
public func onStreamPushStateChanged(_ handler: @escaping (StreamPushState) -> Void)  
public func onMediaRelayStateChanged(_ handler: @escaping (MediaRelayState) -> Void)
```

// SwiftUI - Combine 支持

```
@Published public var messages: [RealtimeMessage] = []  
@Published public var connectionState: ConnectionState = .disconnected  
@Published public var tokenExpirationWarning: TokenWarning? = nil  
@Published public var streamPushState: StreamPushState = .stopped  
@Published public var mediaRelayState: MediaRelayState? = nil  
@Published public var messageProcessingState: ProcessingState = .idle
```

// SwiftUI - AsyncSequence 支持

```
public var messageStream: AsyncStream<RealtimeMessage> { get }
```

```
public var tokenEventStream: AsyncStream<TokenEvent> { get }
public var streamPushEventStream: AsyncStream<StreamPushEvent> { get }
public var mediaRelayEventStream: AsyncStream<MediaRelayEvent> { get }
```

## 性能优化要求

### 1. 内存管理

- 使用 weak 引用避免循环引用
- 及时释放不需要的资源
- 实现对象池减少频繁创建销毁

### 2. 网络优化

- 连接池管理
- 心跳机制
- 断线重连
- 数据压缩

### 3. 线程安全

- 主线程回调 UI 更新
- 后台队列处理网络请求
- 线程安全的状态管理

## 测试要求

### 1. Swift Testing 框架

- 使用 **Swift Testing** 替代 XCTest
- 利用 `@Test` 宏简化测试编写
- 支持参数化测试和条件测试
- 覆盖率 > 80%

### 2. 单元测试

swift

```

import Testing
@testable import RealtimeKit

@Test("RTM Provider Initialization")
func testRTMProviderInit() async throws {
    let config = RTMConfig(appId: "test", appSecret: "secret")
    let provider = AgoraRTMProvider()
    #expect(throws: Never.self) {
        try await provider.initialize(config: config)
    }
}

@Test("Message Sending", arguments: [
    ("text", MessageType.text),
    ("image", MessageType.image),
    ("audio", MessageType.audio)
])
func testMessageSending(content: String, type: MessageType) async throws {
    // 参数化测试用例
}

```

### 3. 集成测试

- Mock 外部 SDK 依赖
- 真实环境端到端测试
- 多服务商切换测试
- 网络异常场景测试
- 使用 `@Test(.disabled())` 标记长时间运行的测试

### 4. 性能测试

- 内存泄漏检测
- CPU 使用率监控
- 网络延迟测试
- 使用 Testing 框架的性能度量工具

## 文档要求

### 1. API 文档

- 完整的 Swift DocC 文档

- 代码示例
- 参数说明

## 2. 使用指南

- 快速开始教程
- 高级用法指南
- 最佳实践

## 3. 迁移指南

- 不同服务商迁移
- 版本升级指南

## 发布计划

### Phase 1: 核心框架 (v0.1.0)

- RealtimeCore 模块
- 基础抽象层
- Mock 测试工具

### Phase 2: 服务商集成 (v0.2.0)

- RealtimeAgora 模块
- RealtimeTencent 模块
- RealtimeZego 模块

### Phase 3: UI 框架支持 (v0.3.0)

- RealtimeUIKit 模块
- RealtimeSwiftUI 模块
- 组件库完善

### Phase 4: 高级功能 (v0.4.0)

- 转推流功能
- Token 自动续期
- 消息处理管道
- 音视频通话

## Phase 5: 企业级功能 (v1.0.0)

- 完整测试覆盖
- 性能优化
- 稳定版本发布

## 开发注意事项

1. **版本兼容性** - 保持向后兼容, 谨慎处理 Breaking Changes
2. **隐私安全** - 符合 App Store 审核要求, 保护用户隐私
3. **许可证管理** - 处理第三方 SDK 的许可证问题
4. **国际化** - 支持多语言错误信息
5. **可观测性** - 内置日志和监控能力
6. **CI/CD** - 自动化测试和发布流程

## 成功标准

1. **易用性** - 5 行代码内完成基本集成
2. **稳定性** - 崩溃率 < 0.01%
3. **性能** - 消息延迟 < 100ms
4. **文档** - 完整的 API 文档和示例
5. **社区** - GitHub Stars > 1000